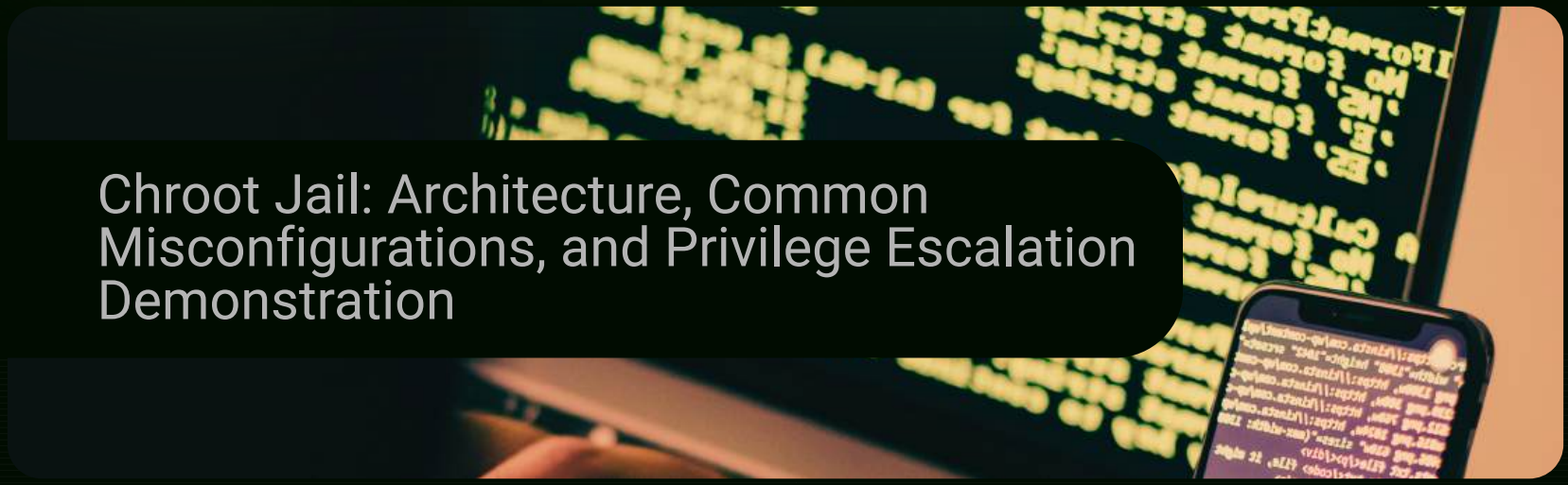
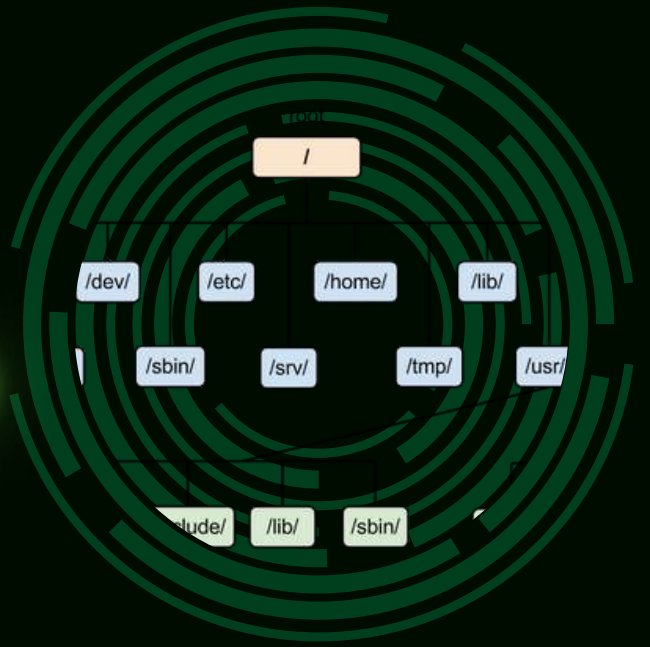
Fundamentals of Cybersecurity

CHROOT



Chroot Jail: Architecture, Common Misconfigurations, and Privilege Escalation Demonstration

WHAT IS A CHROOT JAIL?



- Chroot = change root, a way to make a process see a specific directory as /.
- Introduced in 1979 in Version 7 UNIX
- The process becomes confined to a limited filesystem view a “jail”.
- It does not isolate the kernel, processes, memory, networking, or hardware.
- Created for safe testing, building system trees, and repairing systems.
- Never designed as a security mechanism — just a convenient development tool
- Think of it as a restricted file environment rather than a full security boundary

CHROOT PURPOSE AND USES

Original Purpose

- Safe environment for testing new software
- Install or build operating systems inside a folder
- Provide recovery shells for repairing broken systems
- Build filesystem trees and packages for distributions
- Run programs without risking the real system
- Designed for convenience NOT for security

Modern uses

- Building Linux distribution packages
- Testing or running older software that needs specific libraries
- Rescue environments in Linux installers (live USB repair)
- Light isolation for services like FTP servers
- Training and educational labs

CHROOT VS CONTAINERS

Only isolates filesystem

Shares kernel, processes, and networking with host

Weak isolation.

Use namespaces and cgroups (control groups)

Different views of processes, networks, mounts, users

Much safer and closer to OS-level isolation

SETTING UP A CHROOT JAIL

Building a clean jail on Kali Linux

1. `└─$ sudo apt update && sudo apt upgrade~` update packages

2. `└─$ sudo mkdir -p /var/chroot` create jail directory

3. `└─$ sudo apt install debootstrap` install debootstrap

4. `└─$ sudo debootstrap --variant=minbase kali-rolling /var/chroot http://kali.download/kali`

create a minimal Kali filesystem inside /var/chroot by downloading needed system files from Kali repository.

5.
```
└─$ sudo mount --bind /proc     /var/chroot/proc
└─$ sudo mount --bind /sys      /var/chroot/sys
└─$ sudo mount --bind /dev      /var/chroot/dev
└─$ sudo mount --bind /dev/pts /var/chroot/dev/pts
```

bind essential system directories into the chroot so programs inside it can access processes, devices, and terminals normally.

SETTING UP A CHROOT JAIL

6

```
└─$ sudo chroot /var/chroot /bin/bash
```

Jump into the jail as root

7

```
root@kali:/# useradd -m prisoner
root@kali:/# passwd prisoner
New password:
Retype new password:
passwd: password updated successfully
```

Add low-privilege user and set their password

8

```
└─$ sudo chroot /var/chroot su - prisoner
$
```

Exit then re-enter as prisoner

Now you are a “prisoner” in the “jail, could you break out?

A clean/minimal chroot jail traps a low-privilege user perfectly.

WHY A REGULAR USER CANNOT ESCAPE A CLEAN CHROOT JAIL?

USER IS TRAPPED

A regular user inside a clean chroot jail has no tools that can help them escalate privileges. Everything dangerous is removed: no SUID binaries, no compilers, no interpreters, and no writable system files. They're left with only a shell and a minimal environment, so they can't break out.

MINIMALISM = SECURITY

The isolation doesn't come from chroot itself — it comes from the jail being extremely minimal. With no privilege-escalation paths available, the user stays stuck at their original permission level. The fewer binaries and configs inside the jail, the fewer opportunities to escape

But just because a regular user cannot escape doesn't mean the chroot jail itself is secure. To understand the real limitations, we need to look at what chroot cannot protect

A CHROOT JAIL IS NOT A REAL SECURITY BOUNDARY.

A perfect chroot only isolates the filesystem and still shares the same kernel, system calls, and hardware interface as the host, which means it can stop regular users but cannot contain privileged ones or protect against system-level attacks.

If an attacker ever gains root inside the jail, they can immediately escape using powerful operations like mount, pivot_root, or even a second chroot, and exposed directories such as /proc, /sys, and /dev leak host-level information that helps them map out the system.

On top of that, kernel-level vulnerabilities—like Dirty COW, Dirty Pipe, or StackRot—completely bypass chroot because the jail cannot restrict kernel access, making any compromise of kernel memory instantly collapse the jail's boundaries.

BUT real-world chroot escapes rarely come from deep kernel flaws; they usually come from misconfigurations

HOW MISCONFIGURATIONS BREAK CHROOT

Chroot is only as strong as the environment inside it. Most escapes come from admins accidentally placing dangerous tools or files inside the jail, not from flaws in chroot itself.

Leaving privileged binaries inside the jail

Some programs can grant elevated permissions when misused, and once the attacker gains higher privileges inside the jail, escaping becomes much easier.

Including sensitive or writable system files

If files like password databases or configuration files can be modified, an attacker can change user accounts or system behavior from inside the jail.

Exposing device files

Access to certain /dev entries gives a user strong control over system resources, which can lead to escaping the restricted environment

HOW MISCONFIGURATIONS BREAK CHROOT

Copying interpreters or compilers into the jail

Languages/tools like Python, Bash, or GCC let attackers run complex commands or build tools that help them break out.

Files with dangerous Linux capabilities

If a binary inside the jail has powerful capabilities, it can perform system-level operations that bypass chroot's isolation.

Improper symlinks or inherited file descriptors

If the jail is set up incorrectly, attackers might still reference paths or resources outside the chroot, letting them reach the host filesystem.

CHROOT JAIL ESCAPE TECHNIQUE

Golden Rule of Every Chroot Escape:

1. Gain root inside
2. Break out

Privilege Escalation

- Turning unprivileged user (UID 1000) → root (UID 0)
- On a normal secure system → extremely hard
- Inside a misconfigured chroot → trivial Common paths inside jails:
  1. SUID binaries
  2. Writable /etc/passwd or libraries
- Privilege escalation = the mandatory gateway to freedom

HOW MISCONFIG BREAKS CHROOT: 1) INTERPRETERS/COMPILERS

MISCONFIGURATION:

- Copying python, bash, perl, or gcc into a chroot gives attackers full control tools.

IMPACT:

- Launch a real shell outside the jail
- Load host libraries and escape
- Compile escape programs (gcc)
- Use /proc or ptrace to access the real system

REAL CASES:

- 2019 shared hosting escapes (Python in chroot)
- 2021 GitLab images

WHY ADMINS MISCONFIGURE:

- Convenience: “users need to run scripts”
- Old/legacy docs recommending it
- Misbelief that chroot = strong security

If an interpreter is inside the jail, the jail is already broken.

FILES WITH DANGEROUS LINUX CAPABILITIES

WHAT ARE FILE CAPABILITIES?

Tiny pieces of root-level power attached to an executable.
They let a normal binary perform restricted system operations.
CAP_SYS_ADMIN, CAP_DAC_OVERRIDE, CAP_SYS_CHROOT, CAP_MKNOD.

MISCONFIGURATION

A system administrator accidentally places a binary inside the jail with one of these powerful capabilities still attached.

IMPACT

Capabilities work at the kernel level, not the filesystem level.
Chroot only hides files — it cannot block capabilities.
So the moment that binary runs, it performs real host-level operations from inside the jail.

HOW THE EXPLOIT WORKS

1. Attacker runs the capable binary inside the jail.
2. Kernel sees the capability and grants the privileged operation.
3. Attacker can now:
    - mount the real filesystem, switch root back to /, create device nodes, break out instantly.

REAL-WORLD CASE

CVE-2024-6222 - a Docker Desktop issue where a local container could access the Docker Engine API.

IMPROPER SYMLINKS OR INHERITED FILE DESCRIPTORS

SYMBOLIC LINK (SYMLINK)

- IS A SMALL FILE THAT POINTS TO ANOTHER FILE. INSIDE A CHROOT JAIL, IF IT POINTS OUTSIDE, THE USER CAN STILL ACCESS THAT FILE BECAUSE THE KERNEL FOLLOWS THE PATH WITHOUT CHECKING THE JAIL BOUNDARIES.

INHERITED FILE DESCRIPTORS (FDS)

- FILES OPENED BEFORE ENTERING A CHROOT STAY ACCESSIBLE VIA THEIR FILE DESCRIPTORS (FDS). INSIDE THE JAIL, /PROC/SELF/FD CAN SHOW THESE FDS, WHICH POINT DIRECTLY TO REAL FILES OUTSIDE BECAUSE FDS REFERENCE THE KERNEL, NOT THE FILESYSTEM. ALONG WITH SYMLINKS, THIS ALLOWS BYPASSING THE CHROOT.

LIVE DEMO PREREQUISITES

1 What is Bash (Bourne-Again Shell)?

- Default shell on most Linux systems (/bin/bash)
- Interactive command-line shell and scripting language
- Used to run commands, automate tasks, manage files, and control processes
- Supports history, functions, arrays, job control, and autocomplete

2 What is Dash (Debian Almquist Shell)?

- Lightweight, POSIX-compliant shell, usually at /bin/dash

(POSIX is a set of standards that defines how operating systems and shells should behave)

- Default system shell for scripts (startup, automation) on many Linux systems
- Designed to be fast and minimal uses fewer resources than Bash
- Supports basic shell features, but no interactive extras like history, job control, or arrays

3 find Command

- Standard Unix tool to search for files and directories
- Filters by name, type, size, permissions, etc.
- Can run commands on results using -exec
- Often used to locate SUID binaries, and can be exploitable if find itself has SUID root

THANK YOU

Sama Amr
20235014

Remas Khaled
20235057

Basmala Nabil
20235005